UPI Payments Intelligence Python interview preparation

Prepared for Dhananjay Gupta | Python phase study companion | 27 September 2026

The business questions, Python programs, answers, explanations and limitations behind your completed practice. Use this guide to explain your reasoning, not just memorise outputs.

What this guide is based on

The original UPI_Payments_Intelligence_Project.zip, its Python pipeline, executed notebook, raw snapshots, processed data and tests, plus the lesson code and completion checkpoints visible in our conversation.

Verification: the packaged pipeline was rerun for this guide; all nine tests passed and the learning-assets script executed nine notebook code cells. Numerical answers below are reproduced from that packaged snapshot. External source pages were not independently reverified for this PDF; primary_verified is the package's provenance label.

A clear record of your progress

You completed core Python cleaning, calculations, charts, merchant segmentation, bank analysis, location coverage and the pipeline/test commands. Forecasting was executed by the supplied pipeline, but its detailed study was deferred. Power BI has not yet been completed in this learning sequence.

The separate lesson_01.py and lesson_02.py files are not in the archive and their exact original text is not visible here. Their cleaning and calculation topics are explained using the recovered pipeline; they are not presented as verbatim lesson files. The full recovered pipeline and notebook code are included in the appendices.

How to study

Read sections 1-10 first. Then practise the interview questions aloud and trace the code appendices. For every result, explain its unit, denominator, scope and limitation.

1 | Project purpose and data

Decision: where should a payments operations team investigate processing capacity, merchant activity and technical declines? Python transforms published aggregate snapshots into reproducible evidence for that decision.

Dataset	Rows / period	One row means
Monthly	48; Sep 2022-Aug 2026	One national month
Merchants	29; Aug 2026	One merchant category-month
Banks	12; Aug 2026	One bank-role-month; six per role
Locations	37; Aug 2026	One state/UT or Unclassified entry

Grain means what one row represents. These are aggregate data, not individual payments or customer records. They cannot reveal individual retry journeys, customer retention, fraud cases or technical root causes.

Why each library was used

Library	Use in this project
pandas	Read CSVs, clean columns, handle dates, calculate metrics, group and join tables.
NumPy	Conditional segments, weighted averages, numeric comparisons and supplied forecast maths.
Matplotlib	Two practice trend charts and the supplied dashboard image.
pathlib	Build readable file paths and create output directories.
sqlite3 / unittest	Persist the analytical database and verify important business rules.

Interview answer: I used Python to standardise inconsistent published units, calculate comparable metrics and validate aggregate data before SQL and dashboard reporting.

2 | Importing, cleaning and provenance

Problem solved

Published strings contain units such as million, billion and trillion. Dates need chronological ordering; some percentages contain a Unicode minus sign. Treating these fields as plain strings would make arithmetic or comparisons unreliable.

```
raw = pd.read_csv('data/raw/monthly_reported.csv')
raw['month'] = pd.to_datetime(raw['month_label'], format='%B %Y')
raw['volume_mn'] = raw['volume_reported'].map(pipeline.compact_number) / 1e6
raw['value_cr'] = raw['value_reported'].map(pipeline.compact_number) / 1e7
raw = raw.sort_values('month').reset_index(drop=True)
```

This explanatory extract uses the recovered pipeline helper; it is not the missing original lesson file. read_csv creates a DataFrame. to_datetime makes calendar operations possible. map applies a parser to each value. Sorting must happen before growth or rolling calculations.

Unit conversions

Conversion	Reason
Billion to million: × 1,000	All payment counts use millions.
Trillion rupees to crore: × 100,000	One crore is 10 million rupees.
2.34% to rate: ÷ 100 = 0.0234	Multiplication needs the decimal fraction.

compact_number uses a regular expression to capture the first reported number and unit. This prevents a secondary US-dollar amount in the same source string from being mistaken for the rupee amount.

The pipeline labels 43 older monthly rows secondary_rounded. Five April-August 2026 rows are replaced from official_monthly_overrides.csv and labelled primary_verified. It checks differences against the rounding tolerances before replacement, rather than silently accepting source conflicts.

Missing comparisons and censored values such as <0.01% remain missing numeric values; a true 0% stays zero. Keeping the original label preserves information that a numeric point estimate cannot represent.

3 | Monthly questions and answers

Question	Calculation / answer
What is the latest payment count?	24,508.96 million = 24.50896 billion.
What is the latest transferred value?	2,982,355.95 crore = 29.8235595 lakh crore.
What is the average payment?	10 × value_cr / volume_mn = INR 1,216.84.
How busy is an average day?	24,508.96 / 31 = 790.61 million payments/day.
How did volume change MoM?	(current / previous - 1) × 100 = 3.60%.
How did volume change YoY?	(current / 12 months earlier - 1) × 100 = approximately 22.48%.

```
m['days_in_month'] = m.month.dt.days_in_month
m['daily_volume_mn'] = m.volume_mn / m.days_in_month
m['avg_ticket_inr'] = 10 * m.value_cr / m.volume_mn
m['mom_volume'] = m.volume_mn.pct_change(fill_method=None)
m['mom_daily_volume'] = m.daily_volume_mn.pct_change(fill_method=None)
m['yoy_volume'] = m.volume_mn.pct_change(12, fill_method=None)
m['rolling_3m_volume_mn'] = m.volume_mn.rolling(3, min_periods=3).mean()
```

These are actual calculations in pipeline.py. pct_change returns a decimal fraction: multiply by 100 for percentage display. The first MoM value and first 12 YoY values are missing because no earlier comparison exists. min_periods=3 prevents a partial three-month average.

Important distinction: this Python pipeline smooths monthly volume. Your later SQL Q7 smooths average daily volume. They are different measures and should not be expected to match.

Average ticket is a ratio of total value to total count, not an unweighted mean of monthly averages. Transferred payment value is not company revenue or profit.

4 | Lesson 3: charts and CSV export

Questions: how has daily payment activity changed, and how has the typical payment amount changed? You saved and opened two line-chart PNGs. The conversation confirms an analysis CSV with 48 rows and 13 columns.

```
from pathlib import Path
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

chart_folder = Path('reports/practice')
chart_folder.mkdir(parents=True, exist_ok=True)

def save_chart(column, title, y_label, filename):
    fig, ax = plt.subplots(figsize=(10, 5))
    ax.plot(df['month'], df[column], linewidth=2)
    ax.set_title(title)
    ax.set_xlabel('Month (Sep 2022-Aug 2026)')
    ax.set_ylabel(y_label)
    ax.set_ylim(bottom=0)
    ax.set_xlim(df['month'].min(), df['month'].max())
    ax.grid(axis='y', alpha=0.25)
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
    fig.autofmt_xdate()
    fig.tight_layout()
    fig.savefig(chart_folder / filename, dpi=180)
    plt.close(fig)

save_chart('daily_volume_mn', 'UPI: average daily transactions',
           'Million transactions per day', '01_daily_transactions.png')
save_chart('avg_ticket_inr', 'UPI: average payment amount',
           'INR per payment', '02_average_payment.png')
df.to_csv('data/processed/monthly_analysis_practice.csv', index=False)
```

Compact reconstruction of the chart block shown in the conversation; df is the cleaned monthly analysis DataFrame. The original chart included a source/precision footer. The exact opening portion of lesson_03.py was not available.

A reusable function avoids repeating chart instructions. fig is the whole figure; ax is its plotting area. DateFormatter makes readable dates, savefig writes the image, and close releases memory. index=False avoids exporting an extra DataFrame row-number column.

Finding: daily activity trends upward overall, ending at 790.61 million/day; average ticket trends downward overall, ending at INR 1,216.84. Neither trend must be monotonic, and the charts alone do not establish why it occurred.

5 | Lesson 4: merchant cleaning and shares

Question: which merchant categories handle the most payments? We standardised count units, separated names from MCC codes and calculated each category's share of the observed sample.

```
raw = pd.read_csv('data/raw/merchants_reported.csv')
merchants = raw[['month', 'source_id']].copy()
merchants['month'] = pd.to_datetime(merchants['month'])
names = raw['merchant_reported'].str.rsplit('MCC ', n=1, expand=True)
merchants['merchant'] = names[0].str.strip()
merchants['mcc'] = names[1].str.strip()
volume = raw['volume_reported'].str.split(expand=True)
merchants['volume_mn'] = (
    volume[0].astype(float)
    * volume[1].map({'million': 1, 'billion': 1000})
)
merchants['mom_growth_pct'] = (
    raw['mom_reported'].str.replace('−', '-', regex=False)
    .str.replace('%', '', regex=False).astype(float)
)
merchants['source_precision'] = 'secondary_rounded'
assert not merchants.isna().any().any()
assert not merchants.duplicated(['month', 'mcc']).any()
assert (merchants['volume_mn'] > 0).all()
latest_month = merchants['month'].max()
merchants = merchants.loc[merchants['month'] == latest_month].copy()
merchants['observed_share_pct'] = (
    merchants['volume_mn'] / merchants['volume_mn'].sum() * 100
)
```

rsplit separates at the last MCC marker; strip removes edge whitespace. MCC is an identifier and should stay text. astype(float) enables arithmetic. map performs a unit lookup; an unknown unit becomes missing and is caught by the check.

Denominator: the combined volume of these 29 categories, 12,534.3 million. It is neither all national UPI nor necessarily all merchant payments. In the lesson, mom_growth_pct is in percentage points; the pipeline's mom_volume is a decimal fraction.

6 | Lesson 4: segmentation and outputs

```
median_volume = merchants['volume_mn'].median()
large = merchants['volume_mn'] >= median_volume
growing = merchants['mom_growth_pct'] > 0
contracting = merchants['mom_growth_pct'] < 0
merchants['segment'] = np.select(
    [large & growing, large & contracting,
     ~large & growing, ~large & contracting],
    ['Larger and growing', 'Larger and contracting',
     'Smaller and growing', 'Smaller and contracting'],
    default='Flat'
)
summary = merchants.groupby('segment', as_index=False).agg(
    categories=('mcc', 'count'), volume_mn=('volume_mn', 'sum')
)
output_folder = Path('data/processed')
output_folder.mkdir(parents=True, exist_ok=True)
merchants.to_csv(output_folder / 'merchant_analysis_practice.csv', index=False)
summary.to_csv(output_folder / 'merchant_segments_practice.csv', index=False)
```

The median is 149.6 million payments. With 29 categories, it is the 15th sorted value. At or above the median is our definition of larger. & combines Boolean conditions; ~ reverses them. np.select is similar to SQL CASE WHEN. groupby().agg() creates one summary row per segment.

Segment	Categories	Volume, million
Larger and contracting	3	1185.4
Larger and growing	12	10012.7
Smaller and contracting	6	565.2
Smaller and growing	8	771.0

Finding: three larger categories are contracting and warrant review. Volume and growth together provide more context than either alone. Negative growth does not prove a technical problem.

Code nuance: the practice lesson labels zero growth Flat. The supplied pipeline groups zero growth with growing (>= 0). This snapshot has no zero-growth categories, so the segment counts agree here. Future data could produce different labels unless the rule is standardised.

7 | Merchant findings to explain

Category	Million payments	Observed share
Groceries & supermarkets	4,000.0	31.91%
Fast food restaurants	1,520.0	12.13%
Eating places & restaurants	1,300.0	10.37%
Telecommunication services	817.8	6.52%
Service stations	772.9	6.17%

Answer: groceries and supermarkets rank first: 4,000 million payments and 31.91% of observed category volume. Together the top five represent about 67.10%; that concentration was subsequently calculated explicitly in SQL.

Larger categories with declining volume

Category	Million payments	Reported MoM
Telecommunication services	817.8	-2.40%
Package shops beer, wine and liquor	215.0	-2.30%
Government services not elsewhere classified	152.6	-1.10%

Recommended investigation: compare category activity with earlier complete snapshots, seasonality and merchant operational logs before proposing a cause. This dataset supplies reported growth rates, not enough category-month history to independently reconstruct every growth figure.

Interview answer

I combined category size and reported growth using a median-based rule. I calculated shares within the observed 29 categories, clearly named the denominator, and identified larger shrinking categories for investigation. I did not claim that this sample represented the entire UPI merchant network.

8 | Lesson 5: bank analysis

```
def to_millions(series):
    parts = series.str.split(expand=True)
    return parts[0].astype(float) * parts[1].map(
        {'million': 1, 'billion': 1000})

banks = pd.read_csv('data/raw/banks_reported.csv', parse_dates=['month'])
banks['bank'] = banks['bank'].str.strip()
banks['volume_mn'] = to_millions(banks['volume_reported'])
banks['td_pct'] = banks['td_reported'].str.replace(
    '%', '', regex=False).astype(float)
banks['td_rate'] = banks['td_pct'] / 100
assert banks['volume_mn'].gt(0).all()
assert banks['td_rate'].between(0, 1).all()
assert not banks.duplicated(['month', 'role', 'bank']).any()
banks['estimated_td_mn'] = banks['volume_mn'] * banks['td_rate']
latest_month = banks['month'].max()
sending = banks.loc[(banks['month'] == latest_month)
                    & (banks['role'] == 'Sending')].copy()
assert len(sending) > 0
sending = sending.sort_values('estimated_td_mn', ascending=False)
weighted_td_pct = (sending['estimated_td_mn'].sum()
                   / sending['volume_mn'].sum() * 100)
```

The helper is reused for banks and states. parse_dates loads a date column immediately. The compound bank key includes role because a bank may appear as both sender and receiver. Filtering before aggregation keeps the populations separate.

Answer: Central Bank of India leads the investigation queue: 478.3 million attempts × 0.0234 = 11.19222 million estimated declines, displayed as 11.19 million.

Weighted sample rate: approximately 2.04%. It is total estimated declines divided by total attempts. A simple mean would give small and large banks equal importance.

Technical declines concern system/network issues; these aggregate counts cannot diagnose the specific root cause. Do not sum sending and receiving volumes into a national total: they are different sides of payment processing and the samples are incomplete.

9 | Bank answers and what-if scenario

Sending bank	TD rate	Estimated declines (mn)
Central Bank of India	2.34%	11.19
Baroda U.P. Bank	2.44%	2.68
Punjab and Sind Bank	1.95%	1.50
NSDL Payments Bank Limited	1.17%	1.00
Rajasthan Marudhara Gramin Bank	1.18%	0.93
Maharashtra Gramin Bank	1.23%	0.56

```
benchmark_rate = 0.005
reduction_fraction = 0.50
sending['scenario_avoided_td_mn'] = (
    sending['volume_mn']
    * (sending['td_rate'] - benchmark_rate).clip(lower=0)
    * reduction_fraction
)
```

Scenario question: if the rate above a 0.5% benchmark were halved, how many declines might be avoided at unchanged volume? clip(lower=0) prevents negative reductions for banks already below target.

For Central Bank: 2.34% - 0.50% = 1.84 percentage points of excess. Half is 0.92 percentage points. The scenario rate is therefore 1.42%, not 1.17% and not 0.5%.

Answer: 6.741495 million, approximately 6.74 million, hypothetically avoided declines across the six selected sending banks. This is not an achieved result, a forecast of actual recovery or money saved.

Saved practice outputs: bank_analysis_practice.csv contains all 12 bank-role rows. bank_sending_priority_practice.csv contains the six sending-bank rows with the scenario column. The pipeline uses scenario_avoided_mn for the corresponding metric.

10 | Lesson 5: location coverage

```
states = pd.read_csv('data/raw/states_reported.csv', parse_dates=['month'])
states['state'] = states['state'].str.strip()
states['volume_mn'] = to_millions(states['volume_reported'])
assert states['volume_mn'].gt(0).all()
assert not states.duplicated(['month', 'state']).any()
national = pd.read_csv('data/processed/monthly_cleaned_practice.csv',
                       parse_dates=['month'])
states = states.merge(
    national[['month', 'volume_mn']].rename(
        columns={'volume_mn': 'national_volume_mn'}),
    on='month', how='left', validate='many_to_one')
assert states['national_volume_mn'].notna().all()
states['national_share_pct'] = (
    states['volume_mn'] / states['national_volume_mn'] * 100)
snapshot = states.loc[states['month'] == latest_month].copy()
unclassified = snapshot.loc[snapshot['state'] == 'Unclassified']
assert len(unclassified) == 1
unclassified_pct = unclassified['national_share_pct'].iloc[0]
states.to_csv('data/processed/state_coverage_practice.csv', index=False)
```

latest_month above comes from the bank section of the same lesson; both snapshots are August 2026. In a general refresh, check that their dates agree rather than assuming the bank and location snapshots always align.

A left merge keeps every location row. many_to_one checks that many locations match only one national row for each month. Duplicate national months could otherwise multiply rows and corrupt totals. iloc[0] extracts the one expected Unclassified value.

Answer: 10,880 / 24,508.96 × 100 = 44.39% unclassified. There are 37 location entries, including Unclassified. A substantial part of national volume cannot be assigned geographically in this snapshot.

Unclassified is an explicit category, not a blank to delete. Rounded location rows need not sum exactly to the more precise national total. A residual is a reconciliation finding, not proof of lost payments. Recorded volume is not per-capita adoption.

11 | Pipeline, files and execution

```
python src/pipeline.py
python -m unittest discover -s tests -v
python src/build_learning_assets.py
```

Program / function	Responsibility
clean_data()	Read raw snapshots, parse units, apply overrides, calculate metrics and validate grains.
forecasting()	Run supplied baselines and log-trend evaluation; detailed learning deferred.
warehouse()	Create nine SQLite tables; export bundled SQL results; reconcile average ticket.
figures()	Create reports/dashboard.png.
main()	Coordinate execution, CSV exports, summary, validation report and workbook payload.
build_learning_assets.py	Execute the guided notebook and generate sql/mysql_import.sql.

pipeline.py uses Path(__file__).resolve().parents[1], so paths are anchored to the script's project root. The practice lesson files use relative paths and should be run from the project folder.

The analytical SQLite database is rebuilt by the pipeline; raw CSV snapshots remain unchanged. The separate MySQL export is generated for later import. Rebuilding CSVs does not automatically refresh an existing Excel workbook or Power BI report.

Outputs you should recognise

Practice: monthly_cleaned_practice.csv; monthly_analysis_practice.csv; two practice PNGs; merchant_analysis_practice.csv; merchant_segments_practice.csv; bank_analysis_practice.csv; bank_sending_priority_practice.csv; state_coverage_practice.csv.

Supplied pipeline: data/processed/*.csv; data/upi_intelligence.sqlite; reports/summary.json; reports/validation.json; reports/sql_results/*.csv; reports/dashboard.png; reports/workbook_data.json. Learning assets: notebooks/01_guided_analysis.ipynb and sql/mysql_import.sql.

12 | What the nine tests protect

Test	Business risk prevented
Units and secondary currency	Confusing rupees, crore, million or secondary USD text.
Duplicate keys	Double-counting through repeated records or joins.
Impossible rates	Treating 2.44 as a decimal fraction instead of 2.44%.
Missing / censored values	Replacing unknown or below-threshold values with zero.
Known snapshot control	Unexpected latest month, total volume or override count.
Forecast future isolation	Using future observations in a historical prediction.
Validation-only selection	Choosing a model using the held-out test result.
SQL rate and grain consistency	Different weighted rates in Python and SQL, or broken foreign keys.
Scenario bounds	Negative avoided declines or more avoided than estimated.

Verified result: Ran 9 tests - OK. The learning-assets command also printed: Executed 9 notebook code cells; exported MySQL import.

Tests support computational consistency; they do not prove that external source data are complete, that a scenario will happen, or that every conceivable edge case is covered.

Practice assertions provide quick checks. For durable runtime validation, the pipeline uses explicit ValueError checks in several helpers; Python assertions can be disabled with optimisation. Keep this distinction in mind when explaining production robustness.

Reproducibility

The pipeline records raw-file SHA-256 hashes and library/runtime versions in validation.json. Hashes help identify exactly which input bytes produced a run. A reproducible offline pipeline can be rerun without downloading changing web pages.

13 | Forecasting: supplied, not yet studied

This section documents code you executed through the pipeline. Do not describe forecasting as independently developed or mastered until you can explain and reproduce it.

Model	Idea
daily_naive	Last month's daily rate × next month's days.
seasonal_daily_naive	Same month last year's daily rate × target month's days.
log_trend_12m	Fit a straight line to log daily volume over the last 12 months, then exponentiate and convert to monthly volume.

The first 30 months form initial history; the next six are validation; the last 12 are test. Each one-step prediction uses only earlier months. The winning model is chosen by validation WAPE, then evaluated on the test period.

MAE: mean absolute error in million payments. **WAPE:** total absolute error / total actual volume. These measure prediction error, not guaranteed future accuracy.

Reproduced packaged results: selected model log_trend_12m; test WAPE about 1.12%; September 2026 forecast about 23.99 billion. This is a model output from the historical snapshot, not an observed September result.

The +/-10% range is an assumed planning scenario, not a statistically calibrated prediction interval. The evaluation is retrospective using the current data vintage, and much of its older history is rounded.

Why no random split?

Random splitting can let later observations influence predictions for earlier periods. Time-ordered evaluation better represents the information available at a forecast date. Pipeline tests check temporal cutoffs, but source-vintage revisions remain a separate limitation.

14 | Interview questions: fundamentals

Explain your project in one minute.

I reproduced and studied a Python payments-analysis workflow using 48 national months and aggregate merchant, bank and location snapshots. I standardised dates and units, calculated daily activity and average ticket, segmented merchants, prioritised selected banks by estimated declines and measured unclassified geography. I validated outputs and reconciled Python with SQL. The next phase is Power BI.

What was your contribution?

I ran and practised the guided Python programs, checked the outputs, studied the business calculations and completed the verification steps. The project includes supplied pipeline code. I distinguish what I reproduced from what I independently designed, and I can explain the sections I have studied.

Why use vectorisation?

A column expression applies the operation across the Series without a manual Python row loop. It is concise and well suited to pandas numeric operations. I used it for ticket size, shares and estimated declines.

Why .copy() after filtering?

It creates a separate DataFrame for subsequent changes and makes the intended independence of the filtered result clear.

What is the difference between groupby and a window calculation?

groupby aggregation collapses rows into group summaries. A window-like transform can return group totals alongside each original row. The pipeline uses transform for merchant share denominators.

Why keep missing growth values?

No previous observation means growth is unavailable, not zero. Replacing it with zero would invent evidence of no change.

15 | Interview questions: business reasoning

Why multiply by 10 for average ticket?

Value is in crore rupees and count is in million payments. One crore equals ten million, so the scale factor is 10. The result is rupees per payment.

Why compare daily activity?

Month lengths differ. Dividing by the number of days separates part of the calendar effect from changes in daily processing activity.

Why a weighted bank rate?

The correct combined rate is estimated declines divided by attempts. A simple average gives the same weight to a bank handling 100 attempts and one handling a million.

Why not combine sending and receiving banks?

They describe different roles in payment processing. Adding them can count two sides of activity, and both selected samples are incomplete.

Did your project save 6.74 million payments?

No. It estimated a hypothetical reduction under a 0.5% benchmark and 50% reduction of the excess rate, with unchanged attempt volumes. Actual impact would need an intervention and evaluation.

What does 44.39% unclassified mean?

That share of the national count is in an explicit Unclassified location row. Geographical conclusions are incomplete; it does not mean those transactions failed.

What would you improve next?

Verify and expand official history, obtain complete repeated bank snapshots, preserve source vintages, standardise segmentation edge cases and collect operational evidence to investigate causes.

16 | Revision checklist and source notes

Be able to reproduce these without copying

1. Convert billion payments to millions.
2. Derive average ticket from value and count.
3. Explain the first missing growth values.
4. Build a median-based np.select segment.
5. Name the merchant-share denominator.
6. Compute a weighted decline rate.
7. Explain why clip(lower=0) is needed.
8. Validate a many-to-one merge.
9. Distinguish estimated, hypothetical and observed results.
10. Explain one failing test and why it matters.

Common errors and fixes

Error	First check
FileNotFoundError	Run practice lessons from the extracted project root; confirm paths.
KeyError	Inspect df.columns; practice and pipeline names can differ.
astype(float) fails	Inspect spaces, commas, percent signs and the Unicode minus.
Unexpected NaN	Check unrecognised units, censored rates and missing comparison periods.
Merge multiplies rows	Check uniqueness of the right-hand join key and use merge validation.
Chart uses wrong dates	Convert dates, sort chronologically and check period coverage.

Evidence and provenance

Original archive: UPI_Payments_Intelligence_Project.zip. Internal references: src/pipeline.py, src/build_learning_assets.py, tests/test_pipeline.py, notebooks/01_guided_analysis.ipynb, data/sources.csv, raw/processed CSVs and reports/summary.json. Lesson explanations also use the visible conversation.

The package lists NPCI product statistics (S1), India Dispatch UPI transactions (S2) and India Dispatch payments (S3), accessed 22 September 2026. This guide reproduces the stored data; it does not constitute a fresh audit of those publishers. Source URLs are recorded in data/sources.csv.

Appendix A | Complete recovered pipeline

Source: src/pipeline.py | Original program recovered from the archive. Line wrapping is for PDF readability.

```
"""Rebuild clean data, a SQL warehouse, forecasts and figures from real snapshots.

Run from any directory: python /path/to/project/src/pipeline.py
No network, account, database server or API key is needed for an offline run.
"""
from pathlib import Path
import hashlib
import json
import re
import sqlite3
import sys
import platform

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

ROOT = Path(__file__).resolve().parents[1]
RAW, CLEAN, REPORTS = ROOT/'data/raw', ROOT/'data/processed', ROOT/'reports'
CONFIG = json.loads((ROOT/'config.json').read_text())

def compact_number(value):
    """Parse the first number with its published unit; ignore secondary USD text."""
    match = re.search(r'([0-9][0-9,.]*)\s*(million|billion|trillion)', str(value))
    if not match: raise ValueError(f'Cannot parse a compact number: {value}')
    number = float(match.group(1).replace(',',''))
    return number * {'million':1e6,'billion':1e9,'trillion':1e12}[match.group(2)]

def rate(value):

    text = str(value).strip().replace('−','-').replace(',','')
    if text == 'Not comparable': return np.nan
    if text.startswith('<'): return np.nan  # preserve censoring, never make up a point value
    if not text.endswith('%'): raise ValueError(f'Not a reported percentage: {value}')
    return float(text[:-1])/100

def validate_unique(frame, columns):
    if frame[columns].isna().any().any() or frame.duplicated(columns).any():
        raise ValueError(f'Missing or duplicated key: {columns}')

def validate_rate(values, name):
    if values.isna().any() or not values.between(0,1).all():
        raise ValueError(f'{name} must be a decimal fraction between 0 and 1.')

def clean_data():
    m = pd.read_csv(RAW/'monthly_reported.csv')
    m['month'] = pd.to_datetime(m.month_label,format='%B %Y')
    m['volume_mn'] = m.volume_reported.map(compact_number)/1e6
    m['value_cr'] = m.value_reported.map(compact_number)/1e7
    m['precision'] = 'secondary_rounded'
    m['banks_live'] = np.nan
    m = m[['month','volume_mn','value_cr','banks_live','source_id','precision']].sort_values('month'
    ).reset_index(drop=True)
    official = pd.read_csv(RAW/'official_monthly_overrides.csv',parse_dates=['month'])
    validate_unique(official,['month'])
    for r in official.itertuples():
        mask=m.month.eq(r.month)
        if mask.sum()!=1: raise ValueError('Official override month absent from reported history.')
        original=m.loc[mask].iloc[0]
```

```
        # Published two-decimal billion / trillion display has rounding tolerance.
        if abs(original.volume_mn-r.volume_mn)>5.01 or abs(original.value_cr-r.value_cr)>500.01:
            raise ValueError(f'Source conflict at {r.month.date()}; review rather than silently
    overwrite.')
        m.loc[mask,['volume_mn','value_cr','banks_live','source_id','precision']] =
    [r.volume_mn,r.value_cr,r.banks_live,r.source_id,'primary_verified']

    validate_unique(m,['month'])
    expected=pd.date_range(m.month.min(),m.month.max(),freq='MS')
    if not m.month.equals(pd.Series(expected,name='month')): raise ValueError('Monthly series is not
     contiguous.')
    if (m[['volume_mn','value_cr']]<=0).any().any(): raise ValueError('Nonpositive monthly totals.')
    m['days_in_month']=m.month.dt.days_in_month
    m['daily_volume_mn']=m.volume_mn/m.days_in_month
    m['avg_ticket_inr']=10*m.value_cr/m.volume_mn
    m['mom_volume']=m.volume_mn.pct_change(fill_method=None)
    m['mom_daily_volume']=m.daily_volume_mn.pct_change(fill_method=None)
    m['yoy_volume']=m.volume_mn.pct_change(12,fill_method=None)
    m['rolling_3m_volume_mn']=m.volume_mn.rolling(3,min_periods=3).mean()

    raw=pd.read_csv(RAW/'merchants_reported.csv')
    d=raw.merchant_reported.str.extract(r'^(.*)MCC (\d{4})$')
    merchants=pd.DataFrame({'month':pd.to_datetime(raw.month),'mcc':d[1],'merchant':d[0],
        'volume_mn':raw.volume_reported.map(compact_number)/1e6,
        'value_cr':raw.value_reported.map(compact_number)/1e7,
        'mom_volume':raw.mom_reported.map(rate),'six_month_volume':raw.six_month_reported.map(rate),
        'yoy_volume':raw.yoy_reported.map(rate),'source_id':raw.source_id})
    validate_unique(merchants,['month','mcc'])
    merchants['avg_ticket_inr']=10*merchants.value_cr/merchants.volume_mn
    # The denominator is ONLY the categories in this dataset, not all P2M.
    merchants['observed_volume_share']=merchants.volume_mn/merchants.groupby('month').volume_mn.tran
    sform('sum')
    median=merchants.groupby('month').volume_mn.transform('median')
    merchants['segment']=np.select([
        (merchants.volume_mn>=median)&(merchants.mom_volume>=0),
        (merchants.volume_mn>=median)&(merchants.mom_volume<0),
        (merchants.volume_mn<median)&(merchants.mom_volume>=0)],
        ['Larger and growing','Larger and contracting','Smaller and growing'],default='Smaller and
    contracting')

    raw=pd.read_csv(RAW/'banks_reported.csv')
    banks=pd.DataFrame({'month':pd.to_datetime(raw.month),'role':raw.role,'bank':raw.bank,

        'volume_mn':raw.volume_reported.map(compact_number)/1e6,'td_rate':raw.td_reported.map(rate),
    'source_id':raw.source_id})
    banks['bank_id']=banks.bank.str.lower().str.replace(r'[^a-z0-9]+','_',regex=True).str.strip('_')
    validate_unique(banks,['month','role','bank_id']);validate_rate(banks.td_rate,'Technical decline
     rate')
    banks['estimated_td_mn']=banks.volume_mn*banks.td_rate
    banks['target_td_rate']=CONFIG['scenario_td_target']
    banks['effectiveness']=CONFIG['scenario_effectiveness']
    banks['scenario_avoided_mn']=banks.volume_mn*(banks.td_rate-
    banks.target_td_rate).clip(lower=0)*banks.effectiveness

    raw=pd.read_csv(RAW/'states_reported.csv')
    states=pd.DataFrame({'month':pd.to_datetime(raw.month),'state':raw.state,
        'volume_mn':raw.volume_reported.map(compact_number)/1e6,
        'value_cr':raw.value_reported.map(compact_number)/1e7,
        'reported_volume_share':raw.volume_share_reported.map(rate),
        'reported_volume_share_label':raw.volume_share_reported,'source_id':raw.source_id})
    validate_unique(states,['month','state'])
    states=states.merge(m[['month','volume_mn']].rename(columns={'volume_mn':'national_volume_mn'}),
    on='month',validate='many_to_one')
    states['national_volume_share']=states.volume_mn/states.national_volume_mn
    for frame in (merchants,banks,states):
        if not frame.month.isin(m.month).all(): raise ValueError('Fact month missing from monthly
    dimension.')
        if (frame.volume_mn<=0).any(): raise ValueError('Nonpositive fact volume.')
    return m,merchants,banks,states

def predict_one(history, target_month, model):
    """Only past observations enter any prediction; target month must follow history."""
    h=history.sort_values('month').reset_index(drop=True)
```

```
    target=pd.Timestamp(target_month)
    if target!=h.month.iloc[-1]+pd.offsets.MonthBegin(1): raise ValueError('Expected next contiguous
     month.')
    days=target.days_in_month
    if model=='daily_naive': return float(h.daily_volume_mn.iloc[-1]*days)
    if model=='seasonal_daily_naive':
        row=h[h.month.eq(target-pd.DateOffset(years=1))]
        if len(row)!=1: raise ValueError('Seasonal baseline needs last year.')

        return float(row.daily_volume_mn.iloc[0]*days)
    if model=='log_trend_12m':
        y=np.log(h.daily_volume_mn.tail(CONFIG['trend_lookback_months']).to_numpy())
        x=np.arange(len(y),dtype=float)
        slope,intercept=np.linalg.lstsq(np.column_stack([x,np.ones(len(x))]),y,rcond=None)[0]
        return float(np.exp(intercept+slope*len(y))*days)
    raise ValueError(f'Unknown model: {model}')

def forecasting(m):
    if len(m)<CONFIG['minimum_months_for_forecasting']:raise ValueError('Insufficient forecasting
    history.')
    ntest,nval=CONFIG['forecast_test_months'],CONFIG['forecast_validation_months']
    val_start,test_start=len(m)-ntest-nval,len(m)-ntest
    records=[]
    for model in ['daily_naive','seasonal_daily_naive','log_trend_12m']:
        for i in range(val_start,len(m)):
            pred=predict_one(m.iloc[:i],m.month.iloc[i],model)
            records.append({'month':m.month.iloc[i],'model':model,'split':'validation' if
    i<test_start else 'test',

    'actual_mn':float(m.volume_mn.iloc[i]),'predicted_mn':pred,'train_last_month':m.month.iloc[i-
    1]})
    pred=pd.DataFrame(records)
    pred['absolute_error_mn']=(pred.predicted_mn-pred.actual_mn).abs()
    metrics=pred.groupby(['split','model'],as_index=False).agg(mae_mn=('absolute_error_mn','mean'),a
    bsolute_error_total=('absolute_error_mn','sum'),actual_total=('actual_mn','sum'),months=('month'
    ,'count'))
    metrics['wape']=metrics.absolute_error_total/metrics.actual_total
    selected=metrics[metrics.split.eq('validation')].sort_values(['wape','model']).iloc[0]['model']
    target=m.month.iloc[-1]+pd.offsets.MonthBegin(1)
    estimate=predict_one(m,target,selected)
    next_month=pd.DataFrame([{'month':target,'model':selected,'forecast_mn':estimate,
        'assumed_minus_10pct_mn':estimate*.9,'assumed_plus_10pct_mn':estimate*1.1,
        'range_type':'Assumed +/-10% planning scenarios; not a prediction interval'}])
    return pred,metrics,next_month

def serial_frame(df):
    result=df.copy()

    for col in result.select_dtypes(include=['datetime64[ns]']).columns:result[col]=result[col].dt.s
    trftime('%Y-%m-%d')
    return result

def warehouse(m,merchants,banks,states,metrics):
    frames={
        'sources':pd.read_csv(ROOT/'data/sources.csv'),
        'dim_month':pd.DataFrame({'month':m.month,'year':m.month.dt.year,'month_number':m.month.dt.m
    onth,'days_in_month':m.days_in_month}),
        'dim_merchant':merchants[['mcc','merchant']].drop_duplicates(),
        'dim_bank':banks[['bank_id','bank']].drop_duplicates(),
        'fact_monthly':m.drop(columns=['days_in_month']),
        'fact_merchant':merchants.drop(columns=['merchant']),
        'fact_bank':banks.drop(columns=['bank']),
        'fact_state':states,
        'forecast_metrics':metrics,
    }
    db=ROOT/'data/upi_intelligence.sqlite'
    with sqlite3.connect(db) as con:
        con.execute('PRAGMA foreign_keys=ON')
        # Recreate the disposable analytical database; source CSVs are immutable here.
        for name in ['fact_merchant','fact_bank','fact_state','fact_monthly','forecast_metrics','dim
    _merchant','dim_bank','dim_month','sources']:
            con.execute(f'DROP TABLE IF EXISTS {name}')
        con.executescript((ROOT/'sql/schema_sqlite.sql').read_text())
```

```
        for name,frame in
    frames.items():serial_frame(frame).to_sql(name,con,index=False,if_exists='append')
        if con.execute('PRAGMA foreign_key_check').fetchall():raise ValueError('Foreign key
    validation failed.')
        result_dir=REPORTS/'sql_results';result_dir.mkdir(exist_ok=True)
        text=(ROOT/'sql/analysis_queries.sql').read_text()
        for chunk in re.split(r'(?m)^-- QUERY: ',text)[1:]:
            title,sql=chunk.split('\n',1)
            pd.read_sql_query(sql,con).to_csv(result_dir/(title.strip()+'.csv'),index=False)
        sql_latest=pd.read_sql_query('SELECT 10.0*value_cr/volume_mn AS ticket FROM fact_monthly
    ORDER BY month DESC LIMIT 1',con).ticket.iloc[0]
        if not np.isclose(sql_latest,m.avg_ticket_inr.iloc[-1],rtol=1e-12):raise
    ValueError('SQL/Python ticket reconciliation failed.')
    return frames

def figures(m,merchants,banks,states,pred,next_month,summary):
    plt.rcParams.update({'font.family':'DejaVu
    Sans','font.size':10,'axes.spines.top':False,'axes.spines.right':False,'axes.titleweight':'bold'
    })
    navy,blue,orange='#142B45','#2A70B8','#D97736'
    fig=plt.figure(figsize=(16,10),facecolor='#F7F9FC')
    fig.text(.055,.94,'UPI PAYMENTS INTELLIGENCE',fontsize=23,weight='bold',color=navy)
    fig.text(.055,.902,'August 2026 snapshot  |  National growth, merchant mix and operational
    review',fontsize=12,color='#52647A')
    cards=[('Monthly transactions',f"{summary['latest_volume_bn']:.2f} billion"),('Volume growth
    YoY',f"{summary['latest_yoy']:.1%}"),('Average transfer value',f"INR
    {summary['latest_ticket_inr']:,.0f}"),('Unclassified state
    share',f"{summary['unclassified_share']:.1%}")]
    for i,(label,val) in enumerate(cards):
        x=.055+i*.235;fig.text(x,.837,val,fontsize=23,weight='bold',color=blue);fig.text(x,.802,labe
    l,fontsize=10,color='#52647A')
    gs=fig.add_gridspec(2,2,left=.095,right=.95,top=.73,bottom=.12,hspace=.52,wspace=.37)
    ax=fig.add_subplot(gs[0,0]);ax.plot(m.month,m.volume_mn/1000,color=blue,lw=2.5)
    ax.set_title('National volume (billion
    transactions)',loc='left',pad=12);ax.grid(axis='y',alpha=.15)
    ax.xaxis.set_major_locator(mdates.YearLocator());ax.xaxis.set_major_formatter(mdates.DateFormatt
    er('%Y'))
    ax=fig.add_subplot(gs[0,1]);top=merchants.nlargest(6,'volume_mn').iloc[::-1]
    names={'Groceries & supermarkets':'Groceries','Eating places & restaurants':'Restaurants','Fast
    food restaurants':'Fast food','Telecommunication services':'Telecom','Service stations':'Service
     stations','Bakeries':'Bakeries'}
    ax.barh([names.get(x,x) for x in
    top.merchant],top.volume_mn/1000,color=blue);ax.set_title('Largest observed merchant
    categories',loc='left',pad=12);ax.set_xlabel('Billion transactions; rounded source
    figures');ax.grid(axis='x',alpha=.15)
    ax=fig.add_subplot(gs[1,0]);sample=banks[banks.role.eq('Sending')].nlargest(6,'estimated_td_mn')
    .iloc[::-1]
    labels=[x.replace(' Limited','').replace('Maharashtra Gramin Bank','Maharashtra
    G.').replace('Rajasthan Marudhara Gramin Bank','Rajasthan M.').replace('Central Bank of
    India','Central Bank').replace('Punjab and Sind Bank','Punjab & Sind').replace('NSDL Payments
    Bank','NSDL Payments').replace('Baroda U.P. Bank','Baroda U.P.') for x in sample.bank]
    ax.barh(labels,sample.estimated_td_mn,color=orange);ax.set_title('Estimated TD: six displayed
    sending banks',loc='left',pad=12);ax.set_xlabel('Million attempts; sample only, not national
    failures');ax.grid(axis='x',alpha=.15)
    ax=fig.add_subplot(gs[1,1]);bt=pred[(pred.model.eq(summary['selected_model']))&pred.split.eq('te
    st')]
    ax.plot(bt.month,bt.actual_mn/1000,label='Actual',color=navy,lw=2)
    ax.plot(bt.month,bt.predicted_mn/1000,label='1-month ahead',color=orange,lw=2,linestyle='--')
    ax.set_title(f"Forecast holdout WAPE:
    {summary['test_wape']:.1%}",loc='left',pad=12);ax.set_ylabel('Billion transactions');ax.legend(f
    rameon=False,fontsize=9);ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3));ax.xaxis.se
    t_major_formatter(mdates.DateFormatter('%b %y'));ax.grid(axis='y',alpha=.15)
    fig.text(.055,.04,'Sources: NPCI (latest 5 national months); India Dispatch Research citing NPCI
     (rounded history and ecosystem tables).\nPayment value is money transferred. Bank decline
    counts are estimates. Forecast and improvement scenarios are unverified planning
    outputs.',fontsize=9,color='#52647A')
    fig.savefig(REPORTS/'dashboard.png',dpi=150);plt.close(fig)

def main():
    CLEAN.mkdir(parents=True,exist_ok=True);REPORTS.mkdir(exist_ok=True)
    m,merchants,banks,states=clean_data()
    pred,metrics,next_month=forecasting(m)
    for name,frame in {'monthly':m,'merchants':merchants,'banks':banks,'states':states,'forecast_bac
```

```
    ktest':pred,'forecast_metrics':metrics,'forecast_next_month':next_month}.items():

        serial_frame(frame).to_csv(CLEAN/f'{name}.csv',index=False,float_format='%.10g')
    frames=warehouse(m,merchants,banks,states,metrics)
    # BI import files retain fact/dimension grain and all numeric precision labels.
    for name,frame in
    frames.items():serial_frame(frame).to_csv(CLEAN/f'{name}.csv',index=False,float_format='%.10g')
    model=next_month.model.iloc[0]
    summary={
        'as_of':CONFIG['as_of'],'latest_month':str(m.month.iloc[-1].date()),'history_months':len(m),
        'latest_volume_bn':float(m.volume_mn.iloc[-
    1]/1000),'latest_value_lakh_cr':float(m.value_cr.iloc[-1]/100000),
        'latest_mom':float(m.mom_volume.iloc[-1]),'latest_yoy':float(m.yoy_volume.iloc[-1]),
        'latest_ticket_inr':float(m.avg_ticket_inr.iloc[-
    1]),'daily_volume_mn':float(m.daily_volume_mn.iloc[-1]),
        'unclassified_share':float(states.loc[states.state.eq('Unclassified'),'national_volume_share
    '].iloc[0]),
        'selected_model':model,'test_wape':float(metrics[(metrics.model.eq(model))&metrics.split.eq(
    'test')].wape.iloc[0]),
        'forecast_next_month':str(next_month.month.iloc[0].date()),'forecast_bn':float(next_month.fo
    recast_mn.iloc[0]/1000),
        'observed_merchant_count':len(merchants),'bank_rows':len(banks),'state_rows':len(states),

    'sending_scenario_avoided_mn':float(banks[banks.role.eq('Sending')].scenario_avoided_mn.sum()),
        'top_sending_bank_estimated_declines':banks[banks.role.eq('Sending')].sort_values('estimated
    _td_mn',ascending=False).iloc[0]['bank'],
    }
    (REPORTS/'summary.json').write_text(json.dumps(summary,indent=2))
    figures(m,merchants,banks,states,pred,next_month,summary)
    checks={
        'status':'passed','checks':['unique fact keys','contiguous monthly history','positive
    volumes','rate bounds','official-versus-rounded reconciliation','foreign keys','SQL-versus-
    Python average ticket'],
        'observations':{k:len(v) for k,v in
    [('monthly',m),('merchants',merchants),('banks',banks),('states',states)]},
        'known_limits':['43 historical monthly rows are rounded secondary data','bank sample: six
    displayed banks per role; not all banks','29 merchant categories do not cover all P2M','forecast
     holdout has 12 observations'],
        'raw_sha256':{p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in
    sorted(RAW.glob('*.csv'))},
        'runtime':{'python':platform.python_version(),'pandas':pd.__version__,'numpy':np.__version__
    ,'matplotlib':matplotlib.__version__},
    }
    (REPORTS/'validation.json').write_text(json.dumps(checks,indent=2))
    payload={'summary':summary,'config':CONFIG,'tables':{name:json.loads(serial_frame(df).to_json(or
    ient='records')) for name,df in
    [('monthly',m),('merchants',merchants),('banks',banks),('states',states)]}}
    (REPORTS/'workbook_data.json').write_text(json.dumps(payload,indent=2))
    print(json.dumps(summary,indent=2))

if __name__=='__main__':main()
```

Appendix B | The nine actual tests

Source: tests/test_pipeline.py | Original program recovered from the archive. Line wrapping is for PDF readability.

```
"""Business-risk checks: units, grain, missing values, leakage and SQL consistency."""
import sys
import unittest
from pathlib import Path
import sqlite3
import numpy as np
import pandas as pd

ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/'src'))
import pipeline as p

class TestPipeline(unittest.TestCase):
    @classmethod
    def setUpClass(cls):cls.m,cls.merchants,cls.banks,cls.states=p.clean_data()

    def test_indian_units_and_secondary_currency(self):
        self.assertEqual(p.compact_number('₹29.82 trillion · $311 billion'),29.82e12)
        self.assertAlmostEqual(10*2982355.95/24508.96,1216.8431259425124)

    def test_duplicate_keys_are_rejected(self):
        with self.assertRaises(ValueError):p.validate_unique(pd.DataFrame({'id':[1,1]}),['id'])

    def test_impossible_rates_are_rejected(self):
        with self.assertRaises(ValueError):p.validate_rate(pd.Series([.02,2.44]),'TD')

    def test_missing_and_censored_are_not_zero(self):
        self.assertTrue(np.isnan(p.rate('Not comparable')))
        self.assertTrue(np.isnan(p.rate('<0.01%')))
        self.assertEqual(p.rate('0%'),0)
        self.assertEqual(p.rate('−2.4%'),-.024)

    def test_known_official_control(self):
        r=self.m.iloc[-1]
        self.assertEqual(r.month,pd.Timestamp('2026-08-01'))
        self.assertEqual(r.volume_mn,24508.96)
        self.assertEqual(self.m.precision.eq('primary_verified').sum(),5)

    def test_forecast_does_not_use_future(self):
        first=p.predict_one(self.m.iloc[:36],self.m.month.iloc[36],'log_trend_12m')
        altered=self.m.copy();altered.loc[36:,'daily_volume_mn']=1e9
        second=p.predict_one(altered.iloc[:36],altered.month.iloc[36],'log_trend_12m')
        self.assertEqual(first,second)
        with self.assertRaises(ValueError):p.predict_one(self.m,self.m.month.iloc[-1],'daily_naive')

    def test_selection_is_validation_only(self):
        pred,metrics,nxt=p.forecasting(self.m)
        best=metrics[metrics.split.eq('validation')].sort_values(['wape','model']).iloc[0].model
        self.assertEqual(nxt.model.iloc[0],best)
        self.assertTrue((pred.train_last_month<pred.month).all())
        self.assertEqual(len(pred[pred.split.eq('test') & pred.model.eq(best)]),12)

    def test_sql_weighted_rates_and_grain(self):
        with sqlite3.connect(ROOT/'data/upi_intelligence.sqlite') as con:
            got=pd.read_sql_query('SELECT role,SUM(volume_mn*td_rate)/SUM(volume_mn) AS rate FROM
    fact_bank GROUP BY role',con).set_index('role')['rate']
            for role,g in self.banks.groupby('role'):
                self.assertAlmostEqual(got[role],np.average(g.td_rate,weights=g.volume_mn))
            self.assertEqual(con.execute('PRAGMA foreign_key_check').fetchall(),[])

    def test_scenario_cannot_exceed_estimated_declines(self):
        self.assertTrue((self.banks.scenario_avoided_mn>=0).all())
        self.assertTrue((self.banks.scenario_avoided_mn<=self.banks.estimated_td_mn).all())

if __name__=='__main__':unittest.main(verbosity=2)
```

Appendix C | Learning-assets generator

Source: src/build_learning_assets.py | Original program recovered from the archive. Line wrapping is for PDF readability.

```
"""Create the executed teaching notebook and a MySQL import from the validated DB."""
from pathlib import Path
import contextlib
import io
import json
import sqlite3
import base64
import os

ROOT=Path(__file__).resolve().parents[1]

def mysql_export():
    # Tested source is SQLite. This export targets MySQL 8; no MySQL server is used here.

    keys={'sources':['source_id'],'dim_month':['month'],'dim_merchant':['mcc'],'dim_bank':['bank_id'
    ],

    'fact_monthly':['month'],'fact_merchant':['month','mcc'],'fact_bank':['month','role','bank_id'],
          'fact_state':['month','state'],'forecast_metrics':['split','model']}
    def quote(v):
        if v is None:return 'NULL'
        if isinstance(v,(int,float)):return repr(v)
        return "'"+str(v).replace('\\','\\\\').replace("'","''")+"'"
    parts=['-- Generated from the validated SQLite snapshot. MySQL 8 import; not server-tested.',
      '-- Dedicated database. Existing same-key rows are refreshed; unrelated rows are retained.',
      'CREATE DATABASE IF NOT EXISTS upi_portfolio CHARACTER SET utf8mb4;','USE upi_portfolio;']
    with sqlite3.connect(ROOT/'data/upi_intelligence.sqlite') as con:
        for table,pk in keys.items():
            info=con.execute(f'PRAGMA table_info({table})').fetchall()
            fields=[]
            for _,name,typ,*_ in info:
                t='DOUBLE' if typ=='REAL' else 'INT' if typ=='INTEGER' else 'VARCHAR(1024)' if name
    in ('url','precision_note','coverage') else 'VARCHAR(255)'
                if name=='month':t='DATE'
                fields.append(f'  `{name}` {t}'+(' NOT NULL' if name in pk else ''))
            fields.append('  PRIMARY KEY ('+','.join('`'+k+'`' for k in pk)+')')

            parts.append('CREATE TABLE IF NOT EXISTS `'+table+'` (\n'+',\n'.join(fields)+'\n);')
            cols=[r[1] for r in info];rows=con.execute(f'SELECT * FROM {table}').fetchall()
            assignments=','.join(f'`{c}`=VALUES(`{c}`)' for c in cols if c not in pk)
            parts.append('INSERT INTO `'+table+'` ('+','.join('`'+c+'`' for c in cols)+')
    VALUES\n'+',\n'.join('('+','.join(quote(v) for v in row)+')' for row in rows)+'\nON DUPLICATE
    KEY UPDATE '+assignments+';')
    (ROOT/'sql/mysql_import.sql').write_text('\n\n'.join(parts),encoding='utf-8')

def notebook():
    cells=[]
    def md(text):cells.append({'cell_type':'markdown','metadata':{},'source':text.splitlines(True)})
    def code(text):cells.append({'cell_type':'code','metadata':{},'source':text.splitlines(True),'ex
    ecution_count':None,'outputs':[]})
    md('# UPI Payments Intelligence\nA guided notebook for Dhananjay. Run from top to bottom. The
    included outputs were executed against the packaged CSV snapshot. Source dates and limitations
    are part of the analysis.\n\n**Decision:** Where should a payments operations team investigate
    capacity, merchant activity and technical declines?\n\n**Grain:** one national month, one
    merchant category-month, one bank-role-month, one state-month. There are no individual customer
    transactions.')
    code("from pathlib import Path\nimport sys\nimport pandas as pd\nimport numpy as np\nimport
    sqlite3\nROOT = Path.cwd()\nif ROOT.name == 'notebooks': ROOT = ROOT.parent\nif not (ROOT /
    'src' / 'pipeline.py').exists():\n    raise FileNotFoundError('Open this notebook inside the
    extracted project folder.')\nsys.path.insert(0, str(ROOT / 'src'))\nimport
    pipeline\nprint('Project:', ROOT.name)\n")
    md('## 1. Read and validate\n`read_csv` brings the snapshots into DataFrames. Cleaning
    standardizes dates and units. Validation checks uniqueness before joins, so a repeated key
    cannot silently multiply totals. Five recent national months override rounded values with
    official NPCI figures.')
```

```
code("monthly, merchants, banks, states = pipeline.clean_data()\nprint('Shapes:', {k: v.shape
for k,v in [('monthly',monthly),('merchants',merchants),('banks',banks),('states',states)]})\npr
int(monthly.tail(5).to_string(index=False))\n")
md('## 2. Understand NumPy and vectorization\nA crore is 10 million. Therefore average ticket in
 INR is `10 * value_cr / volume_mn`. Vectorized operations calculate a whole column. `np.select`
 assigns merchant segments using median observed volume and whether reported growth is negative.
 These segments are analyst rules, not source classifications.')
code("manual_ticket = 10 * monthly.iloc[-1]['value_cr'] /
monthly.iloc[-1]['volume_mn']\nprint('Latest average ticket:', round(manual_ticket,
2))\nprint(merchants.groupby('segment').agg(categories=('mcc','count'),
volume_mn=('volume_mn','sum')).to_string())\n")
md('## 3. Ask better growth questions\n`pct_change()` calculates month-over-month change. A
30-day month has fewer processing days than a 31-day month, so compare the daily run rate too.
`pct_change(12)` requires a complete monthly index. Missing first-year comparisons remain
missing.')
code("print(monthly[['month','volume_mn','mom_volume','mom_daily_volume','yoy_volume']].tail(8).
to_string(index=False))\n")
md('## 4. Rank operational review by affected volume\nTechnical decline is a failure associated
with system/network issues. Estimated declines = attempts × TD rate. Use a weighted rate for a
selected sample. The two displayed bank samples are selected and incomplete: neither is a
national estimate, and sending plus receiving must not be added.')
code("sending = banks[banks['role'].eq('Sending')].copy()\nsending =
sending.sort_values('estimated_td_mn', ascending=False)\nprint(sending[['bank','volume_mn','td_r
ate','estimated_td_mn','scenario_avoided_mn']].to_string(index=False))\nprint('Weighted sample
TD:', np.average(sending['td_rate'], weights=sending['volume_mn']))\n")
md('## 5. Join with SQL\nThe pipeline writes a local SQLite database automatically. A bank
dimension holds its name once; fact rows contain its key and measures. MySQL users can import
`sql/mysql_import.sql` into the dedicated `upi_portfolio` database and run the same analysis
queries.')
code("with sqlite3.connect(ROOT / 'data' / 'upi_intelligence.sqlite') as con:\n    q = '''SELECT
 b.bank, f.role, f.volume_mn*f.td_rate AS estimated_td_mn\n           FROM fact_bank f JOIN
dim_bank b ON f.bank_id=b.bank_id\n           WHERE f.role='Sending' ORDER BY estimated_td_mn
DESC'''\n    result = pd.read_sql_query(q, con)\nprint(result.to_string(index=False))\n")
md('## 6. Treat missing geography as a finding\nCompare each state row to the same month’s
national total. Preserve Unclassified. Rounded rows may not add perfectly to the official total;
 a residual is not proof that the network lost transactions.')
code("print(states[states['state'].isin(['Uttar Pradesh','Unclassified'])][['state','volume_mn',
'national_volume_share']].to_string(index=False))\nprint('Rounding residual, million:',
round(states['volume_mn'].sum()-monthly.iloc[-1]['volume_mn'], 3))\n")
md('## 7. Forecast without using future observations\nThree candidates: last-month daily rate,
last-year daily rate, and a 12-month log-linear trend. The first 30 months form the initial
history; the next six select the model; the final 12 evaluate it. Each prediction refits using
earlier months only. WAPE = total absolute error / total actual volume. The ±10% next-month
range is an assumption, not a confidence interval. This is a retrospective exercise using the
current data vintage.')
code("predictions, metrics, next_month = pipeline.forecasting(monthly)\nprint(metrics[['split','
model','months','mae_mn','wape']].to_string(index=False))\nprint(next_month.to_string(index=Fals
e))\n")
md('## 8. Turn findings into actions\nWrite an observation, implication, recommended
investigation and evidence needed. Example: high estimated bank declines justify investigation
of that bank’s error logs. Aggregate data cannot reveal the root cause or prove recoverable
revenue.\n\n**Exercises:** (1) Reproduce the latest ticket without copying the answer. (2)
Explain why a simple average TD is wrong. (3) Change the hypothetical target to 1% and rerun.
(4) Rebuild SQL Q07 yourself. (5) Explain why merchant totals cannot be called total UPI. (6)
Make a baseline forecast manually before reading the trend code.')
code("from matplotlib import pyplot as plt\nimg = plt.imread(ROOT / 'reports' / 'dashboard.png')
\nplt.figure(figsize=(16,10))\nplt.imshow(img)\nplt.axis('off')\nplt.tight_layout()\nplt.show()\
n")
# Execute code cells with standard Python. Capture real text/figure outputs.
namespace={};count=0
import matplotlib.pyplot as plt
previous_cwd=Path.cwd();os.chdir(ROOT)

try:
    for cell in cells:
        if cell['cell_type']!='code':continue
        count+=1;out=io.StringIO();images=[]
        def show(*args,**kwargs):
            buf=io.BytesIO();plt.gcf().savefig(buf,format='png',dpi=90,bbox_inches='tight');imag
es.append(base64.b64encode(buf.getvalue()).decode());plt.close('all')
        old_show=plt.show;plt.show=show
        try:
            with contextlib.redirect_stdout(out):exec(''.join(cell['source']),namespace)
```

```
            finally:plt.show=old_show
            cell['execution_count']=count
            if out.getvalue():cell['outputs'].append({'output_type':'stream','name':'stdout','text':
    out.getvalue().splitlines(True)})
            for im in images:cell['outputs'].append({'output_type':'display_data','metadata':{},'dat
    a':{'image/png':im,'text/plain':['<executed matplotlib figure>']}})
    finally:os.chdir(previous_cwd)
    nb={'cells':cells,'metadata':{'kernelspec':{'display_name':'Python 3','language':'python','name'
    :'python3'},'language_info':{'name':'python','version':'3'}},'nbformat':4,'nbformat_minor':4}
    (ROOT/'notebooks/01_guided_analysis.ipynb').write_text(json.dumps(nb,indent=1),encoding='utf-8')
    print(f'Executed {count} notebook code cells; exported MySQL import.')

if __name__=='__main__':mysql_export();notebook()
```

Appendix D | Executed notebook programs

Source: notebooks/01_guided_analysis.ipynb. All nine code cells are reproduced below in execution order. The setup cell must run first; several later cells require pipeline outputs on disk.

Code cell 1

```
from pathlib import Path
import sys
import pandas as pd
import numpy as np
import sqlite3
ROOT = Path.cwd()
if ROOT.name == 'notebooks': ROOT = ROOT.parent
if not (ROOT / 'src' / 'pipeline.py').exists():
    raise FileNotFoundError('Open this notebook inside the extracted project folder.')
sys.path.insert(0, str(ROOT / 'src'))
import pipeline
print('Project:', ROOT.name)
```

Code cell 2

```
monthly, merchants, banks, states = pipeline.clean_data()
print('Shapes:', {k: v.shape for k,v in
    [('monthly',monthly),('merchants',merchants),('banks',banks),('states',states)]})
print(monthly.tail(5).to_string(index=False))
```

Code cell 3

```
manual_ticket = 10 * monthly.iloc[-1]['value_cr'] / monthly.iloc[-1]['volume_mn']
print('Latest average ticket:', round(manual_ticket, 2))
print(merchants.groupby('segment').agg(categories=('mcc','count'),
    volume_mn=('volume_mn','sum')).to_string())
```

Code cell 4

```
print(monthly[['month','volume_mn','mom_volume','mom_daily_volume','yoy_volume']].tail(8).to_string(
    index=False))
```

Code cell 5

```
sending = banks[banks['role'].eq('Sending')].copy()
sending = sending.sort_values('estimated_td_mn', ascending=False)
print(sending[['bank','volume_mn','td_rate','estimated_td_mn','scenario_avoided_mn']].to_string(inde
    x=False))
print('Weighted sample TD:', np.average(sending['td_rate'], weights=sending['volume_mn']))
```

Code cell 6

```
with sqlite3.connect(ROOT / 'data' / 'upi_intelligence.sqlite') as con:
    q = '''SELECT b.bank, f.role, f.volume_mn*f.td_rate AS estimated_td_mn
           FROM fact_bank f JOIN dim_bank b ON f.bank_id=b.bank_id
           WHERE f.role='Sending' ORDER BY estimated_td_mn DESC'''
    result = pd.read_sql_query(q, con)
print(result.to_string(index=False))
```

Code cell 7

```
print(states[states['state'].isin(['Uttar
    Pradesh','Unclassified'])][['state','volume_mn','national_volume_share']].to_string(index=False)
    )
print('Rounding residual, million:', round(states['volume_mn'].sum()-monthly.iloc[-1]['volume_mn'],
    3))
```

Code cell 8

```
predictions, metrics, next_month = pipeline.forecasting(monthly)
print(metrics[['split','model','months','mae_mn','wape']].to_string(index=False))
print(next_month.to_string(index=False))
```

Code cell 9

```
from matplotlib import pyplot as plt
img = plt.imread(ROOT / 'reports' / 'dashboard.png')
plt.figure(figsize=(16,10))
plt.imshow(img)
plt.axis('off')
plt.tight_layout()
plt.show()
```